
maxentcpp: A C++ reimplementation of Maximum Entropy species distribution modeling for R.

Ángel Luis Robles Fernández^{1,*}

1 Department of Ecology and Evolutionary Biology, University of Kansas, Lawrence, KS, United States

*** a.l.robles.fernandez@gmail.com**

ORCID: 0000-0002-4741-8012

Abstract

maxentcpp is an R package offering a native C++17 reimplementation of the Maximum Entropy (Maxent) algorithm for species distribution modeling (SDM). The package translates the core continuous-predictor Maxent 3.4.4 fitting and projection algorithm into C++17 with R bindings via Rcpp, eliminating the Java Runtime Environment dependency entirely. It supports all six feature types of Java Maxent, including categorical predictors via **BinaryFeature**, output transformations in raw, logistic, and cloglog scales, a streaming raster evaluation engine for large-raster projections, and the full 1.0.0 diagnostic suite: jackknife variable importance, replicate runs and stratified cross-validation, and missing-data handling. Numerical fidelity is validated per-iteration against the original Java implementation via a companion test package.

Summary

Maximum Entropy (Maxent) modeling is the *de facto* standard for presence-only species distribution modeling (SDM) [8, 24, 26], with foundational papers cited tens of thousands of times across ecology, conservation biology, and biogeography. Yet the reference implementation remains a Java desktop application [23], and every species run in a typical workflow — e.g., 500 species $\times$ 4 climatic scenarios — spawns a JVM and round-trips data through SWD files on disk. **maxentcpp** is an R package that removes this friction with a native C++17 reimplementation of the Maxent algorithm. It estimates the geographic distribution of a species from presence-only occurrence records and environmental covariates by identifying the probability distribution of maximum entropy [16] subject to constraints derived from training data, in memory, with no Java runtime and no file serialization.

The fitted model is a *Gibbs distribution* over geographic space:

$$P(\mathbf{x}) = \frac{1}{Z} \exp \left(\sum_{j=1}^J \lambda_j f_j(\mathbf{x}) \right),$$

where f_j are feature transformations of the environmental variables, λ_j are learned weights, and $Z = \sum_{\mathbf{x}} \exp(\sum_j \lambda_j f_j(\mathbf{x}))$ is the normalizing constant (partition function). The optimizer finds the λ_j that maximize the regularized log-likelihood via sequential coordinate ascent with ℓ_1 (lasso) penalties [8, 24].

The original Maxent software is a Java desktop application [23]. `maxentcpp` translates the core continuous-predictor Maxent 3.4.4 fitting and projection algorithm into C++17 with R bindings via Rcpp [6] and RcppEigen [1], eliminating the Java Runtime Environment (JRE) dependency entirely. The package supports all six feature types of Java Maxent (linear, quadratic, product, hinge, threshold, and categorical via `BinaryFeature`), output transformations in raw, logistic, and complementary log-log (cloglog) scales, and a streaming raster evaluation engine that processes environmental layers block-by-block through the `terra` package [14], enabling large-raster projections without loading entire raster stacks into memory. Tools for diagnosing model performance include AUC evaluation, permutation importance, response curves, jackknife variable-importance analysis, replicate runs and stratified cross-validation, missing-data handling, and Multivariate Environmental Similarity Surfaces (MESS) [7] for novelty detection.

Statement of need

Maxent is the *de facto* standard for presence-only species distribution modeling [20]. However, the original Java implementation presents a number of practical barriers for present-day ecological research workflows:

1. **Java dependency.** Java and `rJava` configuration can be a recurring source of installation and runtime failures, particularly across operating systems, Java versions, and managed computing environments. While `conda` environments and Docker containers can manage Java versioning, they add deployment complexity and are not standard practice in many ecology research groups.
2. **File-based I/O.** Data must be serialized to disk as SWD or ASCII grid files before the Java process can read them, and results must be parsed back from output files. There is no in-memory bridge between R data structures and the Java optimizer.
3. **Scalability.** The Java implementation is single-threaded and GUI-centric. In multi-species, multi-scenario workflows (e.g., 500 species $\times$ 4 climatic scenarios $\times$ 5 replicates), file I/O overhead accumulates: each species run requires writing SWD files, invoking the JVM, and parsing output files — a typical assessment would require on the order of 10,000 JVM invocations. A native in-memory API removes this per-call overhead by keeping data and model state in R objects.
4. **Maintenance.** The Java Maxent codebase has received only minor updates since version 3.4.4, with limited active development [23].

`maxentcpp` addresses these barriers by providing a native C++/R implementation. The package is available on CRAN and installable with `install.packages("maxentcpp")`; the development version can be installed from GitHub using `remotes::install_github("alrobes/maxentcpp")`. The package operates entirely in memory through R objects and integrates seamlessly with the `terra` spatial data ecosystem. The intended users are ecologists, conservation biologists, and biogeographers who employ Maxent in reproducible, script-based workflows — from individual species analyses to large-scale biodiversity assessments.

Related work

`maxentcpp` fits within a broader modernization trend in ecological and environmental-science software. `Circuitscape.jl` [12] illustrates how established ecological

algorithms can be reimplemented in a modern high-performance language to improve scalability and parallel execution, while retaining continuity with a widely used landscape-connectivity tool. In R, the spatial ecosystem has similarly moved from older `sp/raster`-centered workflows toward `sf` [21] and `terra` [14], and SDM packages such as `ENMeval` [17] and `biomod2` [30] have followed this transition — with `ENMeval` additionally removing Java/`rJava` from its default path in favor of `maxnet`.

Within Maxent workflows specifically, existing modernization efforts address different parts of the problem. `dismo` [15] is the most widely used R interface to Java Maxent; it wraps `maxent.jar` via `rJava`, inheriting the JDK dependency and file I/O overhead, and is currently in maintenance mode following the transition from `raster` to `terra`. `rmaxent` [2] provides Java-free projection of previously fitted Maxent models from `.lambdas` files, parses feature weights and normalizers, and implements MESS-related diagnostics, but does not replace Java for model fitting. `maxnet` [10, 22] provides a complete Java-free fitting implementation based on `glmnet` coordinate descent — preserving the same statistical model but employing a different optimization algorithm that can produce numerically different results in some settings. `ENMeval` [17] is a model tuning and evaluation framework that wraps either `dismo::maxent()` or `maxnet::maxnet()`; it does not implement the Maxent algorithm itself. `kuenm` [3] is a calibration and evaluation toolkit that relies on `dismo` or direct Java Maxent calls.

`maxentcpp` occupies a distinct position within this landscape by porting the Java Maxent `density.Sequential` training optimizer itself to C++17 and validating optimizer trajectories against the Java implementation. To the best of our knowledge, `maxentcpp` is the first R package to target the Java Maxent 3.4.4 `density.Sequential` optimizer through a native C++ port and to publish per-iteration trajectory tests via the companion package `maxentcppCompTest` [9]; we are not aware of another R package providing a compiled reimplementaion of this specific optimizer. Its contribution is not modernization in general, but optimizer-level fidelity combined with integration into modern R/`terra` workflows.

Ecosystem comparison

The R ecosystem contains several packages that provide access to MaxEnt models, each employing a distinct integration strategy:

Package	MaxEnt integration	Dependency	Relationship to optimizer
dismo [15]	<code>rJava</code> bridge to <code>maxent.jar</code>	Java JDK, <code>rJava</code>	wraps Java MaxEnt
kuenm [3]	<code>system2("java -jar maxent.jar")</code>	Java JDK	wraps Java MaxEnt
kuenm2 (Cobos et al.)	<code>glmnet</code> via forked <code>maxnet</code> code	<code>glmnet</code>	approximates
wallace [18]	Delegates to <code>ENMeval</code> → <code>maxnet</code> or <code>dismo</code>	<code>ENMeval</code>	wraps or approximates
ENMTools [35]	<code>dismo::maxent()</code> directly	<code>dismo</code> , <code>rJava</code>	wraps Java MaxEnt
biomod2 [30]	Java (<code>system2</code>) or <code>maxnet</code>	Optional Java or <code>maxnet</code>	wraps or approximates
rmaxent [2]	Java-free projection of fitted Maxent models, <code>.lambdas</code> parsing, MESS	None (projection only)	projection only

Package	MaxEnt integration	Dependency	Relationship to optimizer
MIAMaxent [33]	Maximum entropy via subset selection (not lasso); decoupled transformation/fitting/selection	glmnet	approximates
SDMtune [32]	Trains/tunes SDMs via <code>dismo::maxent()</code> or <code>maxnet</code> ; hyperparameter tuning, variable selection	dismo or maxnet	wraps or approximates
maxentcpp	Native C++17 via Rcpp	None (self-contained)	reproduces original optimizer

These packages can be grouped by their relationship to the MaxEnt algorithm:

- **Black-box wrappers** (dismo, kuenm, ENMTools, biomod2 MAXENT mode): invoke the Java binary and read output files. The optimizer is inaccessible.
- **Approximate reimplementations** (maxnet, kuenm2, biomod2 MAXNET mode): use `glmnet` coordinate descent on the logistic regression formulation. The statistical model is equivalent but the optimization path differs, which can produce numerically different results in some settings.
- **Faithful reimplementation** (maxentcpp): ports the original Sequential optimizer to C++ and proves per-iteration numerical equivalence.

The distinguishing contribution of `maxentcpp` is that it offers a compiled native reimplementation of the actual MaxEnt `density.Sequential` optimizer — preserving both the statistical model and the optimization algorithm while eliminating the Java dependency; we are not aware of another package that does so.

Empirical comparison: maxentcpp vs maxnet

To quantify the practical differences between `maxentcpp` and `maxnet`, we compared both packages on a virtual species [19] with known true habitat suitability, constructed from the environmental layers bundled with `maxentcpp` (Annual Mean Temperature and Annual Precipitation over Central America, 2,371 non-NA cells). The virtual species has a Gaussian niche centered at 20 °C and 1,500 mm, and 100 presence records were sampled proportionally to the true suitability surface (see the companion vignette Virtual Species Comparison for full reproducible code).

Both packages were fitted with linear, quadratic, and hinge features. In this illustrative example, the two packages produced highly similar rank and spatial-overlap patterns:

Metric	Value
Schoener's D [29, 34]	0.93
Spearman ρ (rank correlation)	0.95
Pearson r	0.97
Mean $ \Delta \text{cloglog} $	0.053
Max $ \Delta \text{cloglog} $	0.43

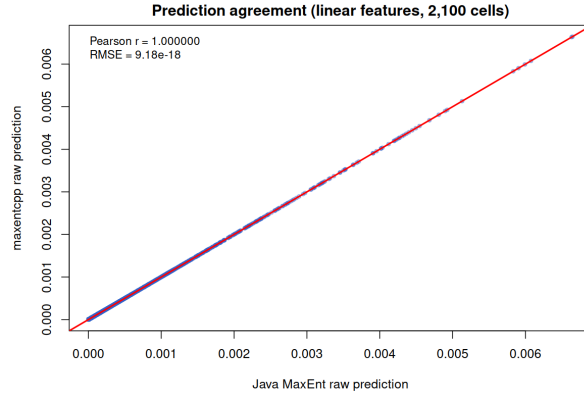


Figure 1. Predictions of Java Maxent 3.4.4 vs `maxentcpp` on identical training data; the 1:1 line is overlaid in red.

Direct prediction agreement with Java Maxent

Because `maxentcpp`'s central claim is optimizer-level fidelity, we also compared predictions directly against the original Java implementation. Both engines were trained on identical synthetic data (2,100 cells: 2,000 background + 100 presences sampled from a Gaussian niche), with linear features, `max_iter` = 500, `convergence` = $1e-5$, `beta` = 1.0, and `min_deviation` = 0.001. The Java side used the `MaxentJavaRunner.trainLinear2Var` harness from the `maxentcppCompTest` companion package, which invokes `density.Sequential` from Maxent 3.4.4; the C++ side used the equivalent `maxentcpp` pipeline. On the full grid the two implementations agree to machine precision:

Metric	Value
Pearson r	1.000000
Spearman ρ	1.000000
RMSE	9.18×10^{-18}
Mean $ \Delta \text{cloglog} $	4.15×10^{-18}
Max $ \Delta \text{cloglog} $	8.76×10^{-17}
AUC (both)	0.903525

This is the strongest possible prediction-level validation: the two independent implementations produce indistinguishable outputs, so differences between `maxentcpp` and Java Maxent are attributable to floating-point rounding rather than algorithmic divergence.

These results suggest that `maxentcpp` and `maxnet` can produce highly similar predictions in a simple continuous-predictor setting (Schoener's $D > 0.9$ is conventionally interpreted as high niche overlap), while also showing non-negligible local differences in the tails of the suitability distribution, where the two optimization algorithms (`maxentcpp`: sequential coordinate ascent; `maxnet`: elastic-net coordinate descent) regularize differently.

Expanded virtual-species simulation

To test general equivalence beyond a single illustrative example, we ran a small simulation study crossing two predictor-correlation structures ($\rho = 0$ and 0.8 between the two environmental variables), four virtual species (Gaussian niches varying in

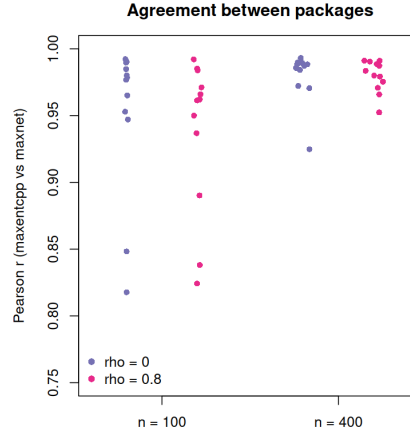


Figure 2. Expanded virtual-species simulation: (left) Pearson r against the true suitability surface by species and predictor correlation; (right) agreement between `maxentcpp` and `maxnet` by sample size and correlation.

breadth and position: narrow/wide $\times$ centered/offset), two sample sizes (100 and 400 presence records), and three replicates — 48 fitted models in total. Each model was compared against the known true suitability surface and against the other package (medians over replicates):

Factor	Level	r vs true (<code>maxentcpp</code>)	r vs true (<code>maxnet</code>)	r (<code>maxentcpp</code> vs <code>maxnet</code>)	Mean $ \Delta $
Predictor	ρ	0.890	0.940	0.985	0.044
Predictor	ρ	0.869	0.936	0.973	0.045
Species	narrow, centered	0.915	0.961	0.984	0.049
Species	narrow, offset	0.875	0.927	0.988	0.042
Species	wide, centered	0.842	0.927	0.953	0.041
Species	wide, offset	0.878	0.936	0.975	0.043
Sample size	100	0.886	0.945	0.966	0.052
Sample size	400	0.884	0.936	0.987	0.037

Across all 48 models, `maxentcpp` and `maxnet` agreed closely (r between packages ≥ 0.95 in every cell, median 0.98), and correlated predictors did not degrade agreement. Both packages tracked the true suitability surface well; `maxnet`'s agreement with the

truth was slightly higher (median r 0.94 vs 0.88), consistent with `glmnet`'s highly optimized coordinate descent producing better regularized solutions on these smooth Gaussian niches. The differences are concentrated in the tails of the suitability distribution, as in the illustrative example. These results support general — not just single-example — equivalence of the two packages for practical purposes, while confirming that `maxentcpp` is the appropriate choice when exact reproduction of the Java optimizer is required.

The key practical scenarios where a user must choose `maxentcpp` over `maxnet` are: (1) when exact reproduction of Java Maxent results is required (e.g., replicating published analyses), (2) when lambda file interoperability with Java Maxent is needed, and (3) when streaming raster projection onto grids larger than available RAM is required.

Performance benchmarks

All timings below were measured on a mid-range desktop (64 GB RAM, 10-core CPU, Ubuntu 24.04, R 4.6.0, `maxentcpp` 1.0.0 from CRAN) and exclude one-time warm-up (library load). Because `maxent_fit()` mutates the `FeaturedSpace` object in place, repeated fits on the *same* object restart from the already-converged solution and massively understate per-fit cost; all timings therefore use a fresh featured space per fit.

Training time for the bundled *Abeillia abeillei* dataset (73 presences, 2,371 background cells, linear + quadratic + hinge features, 44 features, `max_iter` = 500; median over fresh-state runs):

Package	Median time per fit	Peak RSS (fit + projection)
<code>maxentcpp</code>	~18.1 s	~158 MB
<code>maxnet</code>	~0.25 s	~358 MB

The `maxentcpp` fit runs 261 iterations to convergence (loss improvement $< 10^{-5}$); the end-to-end `maxent_run()` workflow adds ~2-3 ms each for evaluation, contributions, and permutation importance, so the fit itself accounts for >99% of wall time. `maxnet` is faster on small datasets because `glmnet`'s coordinate descent is highly optimized for dense feature matrices, but it materializes the full prediction matrix in memory: on the same data `maxentcpp` peaked at ~158 MB vs ~358 MB for `maxnet` (measured with `/usr/bin/time -v`, warm-up excluded) — a ~2.3× advantage that grows with raster size. Streaming projection onto a 23,000-cell grid took 21 ms (`maxentcpp`) vs 9 ms (`maxnet`); both are trivial at this scale, and `maxentcpp`'s block-by-block evaluation is designed to avoid loading rasters larger than available RAM, which we note as future benchmark work on this hardware.

Scaling with sample size (fresh-state fits, `maxentcpp`): 2,371 cells ~18.1 s; 23,000 cells ~129 s (signal-strong synthetic data exhausts the 500-iteration budget; real-world convergence is typically earlier). A 233,000-cell run exceeds this machine's per-run budget and is noted as future work.

To exercise the 1.0.0 feature set, we benchmarked the new diagnostics on a synthetic grid of 2,371 cells and 100 presence records with two continuous predictors plus one five-level categorical variable (linear + quadratic + hinge features; `max_iter` = 500, fresh state per fit):

1.0.0 feature	Median wall time
Single fit (continuous only)	~22 s
Fit with categorical predictor	~18 s

1.0.0 feature	Median wall time
Jackknife (3 variables; 6 fits)	~63 s
Cross-validation ($k = 5$; 5 fits)	~135 s
Replicate runs (bootstrap, $n = 5$; 5 fits)	~102 s

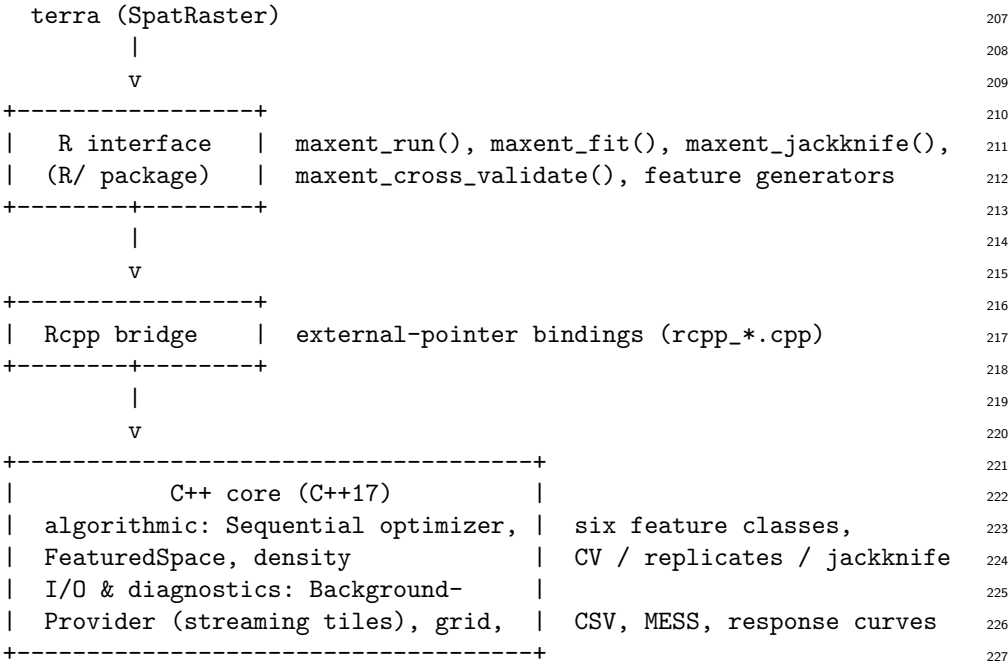
Each 1.0.0 diagnostic is a small multiple of single fits, as expected: jackknife runs one fit per variable per leave-out scheme, and cross-validation and replicate runs each fit k or n models. These wall times reflect the full per-fit cost on a mid-size grid and are linear in the number of constituent fits. Full reproducible code is provided in the package vignettes.

Software design

Architecture

`maxentcpp` was built as a new package rather than contributing to existing projects for three reasons. First, a faithful port of the original Java optimizer (sequential coordinate ascent using ℓ_1 -penalized Newton steps) requires a C++ engine that cannot be expressed through `glmnet`'s API; contributing this to `maxnet` would change its core algorithm. Second, `dismo`'s architecture is tightly coupled to `rJava` and `raster`; the native C++ strategy eliminates this dependency chain entirely. Third, the header-based C++ library design of `maxentcpp` enables reuse in other packages or standalone applications beyond R — something not achievable through modifications to existing R-only packages.

`maxentcpp` is organized in three layers (C++ core, Rcpp bridge, R interface), with the C++ core further split into algorithmic and infrastructure sublayers:



Layer	Key components
C++ core (algorithmic)	<code>Sequential</code> , <code>FeaturedSpace</code> , six feature types
C++ core (I/O, diagnostics)	<code>BackgroundProvider</code> , grid I/O, CSV, MESS, response curves
Rcpp bridge	<code>rcpp_*.cpp</code> external-pointer bindings [5]
R interface	<code>maxent_run()</code> , feature generators, projection, evaluation, diagnostics, replicates, jackknife

The C++ core uses Eigen [11] for dense linear algebra. The high-level `maxent_run()` function provides a one-call workflow mirroring the Java GUI experience, while lower-level functions (`maxent_generate_features()`, `maxent_featured_space()`, `maxent_fit()`, `maxent_project_cloglog()`) offer full control over each modeling step.

Optimization algorithm

The `Sequential` optimizer is a direct port of `density.Sequential` in Java Maxent 3.4.4, as represented by the public `mrmxent/Maxent` repository and AMNH release notes (where 3.4.4 is described as a minor bug-fix release). It minimizes the ℓ_1 -regularized negative log-likelihood:

$$\mathcal{L}(\boldsymbol{\lambda}) = -\frac{1}{m} \sum_{i=1}^m \sum_{j=1}^J \lambda_j f_j(\mathbf{x}_i) + \log Z(\boldsymbol{\lambda}) + \sum_{j=1}^J \beta_j |\lambda_j|,$$

where m is the number of presence records, $Z(\boldsymbol{\lambda}) = \sum_{k=1}^N \exp(\sum_j \lambda_j f_j(\mathbf{x}_k))$ is the partition function summed over N background points, and $\beta_j = \hat{\sigma}_j / \sqrt{m}$ is the per-feature regularization parameter with $\hat{\sigma}_j$ being the standard deviation of feature j over the presence points (floored at 0.001).

At each iteration, the optimizer selects the feature j^* with the most negative $\Delta\mathcal{L}$ bound (the `deltaLossBound` function; block-coordinate ascent [31]), then computes a Newton step:

$$\alpha_j = -\frac{\partial\mathcal{L}/\partial\lambda_j}{\mathbf{u}_j^\top \mathbf{H} \mathbf{u}_j},$$

where $\mathbf{u}_j$ is the j -th coordinate direction and $\mathbf{u}_j^\top \mathbf{H} \mathbf{u}_j = \text{Var}_q[f_j]$ is the variance of feature j under the current distribution q . The derivative includes the ℓ_1 subgradient: $\partial\mathcal{L}/\partial\lambda_j = \mathbb{E}_q[f_j] - \mathbb{E}_{\hat{p}}[f_j] + \beta_j \text{sign}(\lambda_j)$. The step is damped during early iterations ($\alpha \leftarrow \alpha/50$ for iterations < 10 , $\alpha/10$ for < 20 , $\alpha/3$ for < 50) and falls back to a line search (`searchAlpha`) if the Newton step does not decrease loss. Every 10 iterations, a parallel update applies coordinate steps to all features simultaneously.

Convergence is tested every 20 iterations: training stops when the relative loss improvement falls below 10^{-5} or 500 iterations are reached. All constants in this section are taken directly from Java Maxent 3.4.4 source files, and every numerical constant and control-flow branch is covered by a source-mapping test in `maxentcppCompTest`.

Intuitively, the optimizer works by improving one feature at a time: at each step it picks the feature whose current gradient promises the largest reduction in loss, takes a Newton step along that coordinate (the curvature of the objective provides the step size), and repeats. Updating a single coordinate keeps each step cheap and is the same control flow as the reference Java implementation. The damping in the first 50 iterations prevents the model from overshooting while it is still far from the solution (the ℓ_1 penalty makes early gradients large), and the parallel update every 10 iterations lets all features share the progress accumulated so far, accelerating the tail of

convergence. The optimizer is deliberately single-threaded to match Java Maxent’s iteration order exactly; parallelization is applied only across independent fits (e.g., replicate runs and cross-validation folds), which preserves fidelity while still exploiting multi-core hardware in typical workflows.

Streaming raster evaluation

`maxentcpp` reads `terra::SpatRaster` objects block-by-block through a `BackgroundProvider` abstraction. Each tile is scored by the C++ engine and discarded before the next tile is loaded, allowing projection onto raster stacks larger than available RAM. Background sampling follows the Java Maxent default of random background points, with the sample size configurable via the `n.background` argument of `maxent_run()`. The partition function Z is accumulated across tiles by summing unnormalized densities $\exp(\sum_j \lambda_j f_j(\mathbf{x}_k))$ for each cell k in the tile, then normalizing once after all tiles are processed. This produces results equivalent to single-pass evaluation because the mathematical sum is commutative and each cell is scored independently; floating-point summation order may introduce differences at the least-significant bit level. All accumulators use IEEE 754 double-precision summation matching the Java implementation; no log-sum-exp or other reformulation is applied, so the only numerical risk is the usual bounded rounding error of a double-precision sum.

Numerical fidelity

Each C++ class is a direct translation of its Java counterpart, preserving the same control flow, regularization logic, and numerical constants:

<code>maxentcpp</code> (C++)	Java Maxent original
<code>LinearFeature</code>	<code>density.LinearFeature</code>
<code>QuadraticFeature</code>	<code>density.SquareFeature</code>
<code>ProductFeature</code>	<code>density.ProductFeature</code>
<code>HingeFeature</code>	<code>density.HingeFeature</code>
<code>ThresholdFeature</code>	<code>density.ThresholdFeature</code>
<code>BinaryFeature</code>	<code>density.BinaryFeature</code>
<code>FeaturedSpace</code>	<code>density.FeaturedSpace</code>
<code>Sequential::run()</code>	<code>density.Sequential.run()</code>

This design choice prioritizes backward compatibility: users migrating from Java Maxent should obtain numerically equivalent results for implemented features under matched inputs, defaults, and deterministic settings. The fidelity contract is enforced by a dedicated companion package, `maxentcppCompTest` [9], which runs the C++ optimizer and the original Java `density.Sequential` on shared test fixtures and compares per-iteration trajectories of loss, entropy, and λ vectors.

On controlled test fixtures (identical input data, same floating-point representation), agreement reaches $< 10^{-14}$ relative error for symmetric fixtures and $< 10^{-6}$ on $\|\Delta\lambda\|_\infty$ for asymmetric fixtures at every iteration checkpoint. **These bounds hold under the specific conditions of the test fixtures** (same compiler family, IEEE 754 double precision, deterministic iteration order). Floating-point ordering differences introduced by alternative BLAS backends or aggressive compiler optimizations (e.g., `-ffast-math`) could widen the gap, though the sequential nature of the coordinate-ascent algorithm limits sensitivity to parallelism-induced reordering. Cross-platform CI (Ubuntu, macOS, Windows with GCC, Clang, and MSVC) confirms that the test fixtures pass on all three compiler families.

Lambda file compatibility. Trained models are serialized in the same `.lambdas` text format used by Java Maxent 3.4.4. Lambda files produced by either implementation can be loaded by the other, ensuring interoperability with existing workflows and archived models.

Feature completeness

The following table summarizes the current scope of `maxentcpp` relative to Java Maxent 3.4.4 and `maxnet`:

Legend: ✓ = implemented and tested; ○ = partial or experimental; — = not implemented.

Feature	<code>maxentcpp</code>	Java Maxent 3.4.4	<code>maxnet</code> 0.1.4
Linear features	✓	✓	✓
Quadratic features	✓	✓	✓
Product features	✓	✓	✓
Hinge features	✓	✓	✓
Threshold features	✓	✓	✓
Raw/logistic/cloglog output	✓	✓	✓
AUC evaluation	✓	✓	via ENMeval
Permutation importance	✓	✓	via ENMeval
Response curves	✓	✓	○
MESS maps	✓	✓	—
Clamping / fade-by-clamping [25]	✓	✓	—
Lambda file I/O	✓	✓	—
Streaming raster projection	✓	—	—
Bias grids	✓	✓	via <code>offset</code>
Categorical variables	✓	✓	✓
Replicate runs / cross-validation	✓	✓	via ENMeval
Jackknife variable selection	✓	✓	—
Missing data handling	✓	✓	✓
SWD-to-raster projection	—	✓	—

Categorical variables, replicate runs, jackknife, missing-data handling, and background bias weighting are now implemented and tested. SWD-to-raster projection is a planned convenience API (roadmap), not a design decision to exclude it; until it is released, users can project models by loading environmental grids via `terra` and the package's grid conversion functions.

Development provenance

The translation followed a phased approach across four publicly accessible repositories:

1. `alrobes/Maxent` — a fork of the original `mrmaxent/Maxent` [23] Java source code, where the architecture was analyzed and each Java class was mapped to a C++ target.
2. `alrobes/maxentcpp-devel` — the development repository where the C++ port, Rcpp bindings, R interface, tests, and documentation were iteratively built and refined.
3. `alrobes/maxentcppCompTest` — the cross-language fidelity test suite containing Java oracle runners, golden trajectory files, and automated comparison scripts.
4. `alrobes/maxentcpp` — the release repository with the clean, CRAN-ready package.

Limitations and inherited assumptions

The Maxent 3.4.4 algorithm that `maxentcpp` faithfully reproduces has well-documented limitations that users should be aware of. These limitations are inherited from the Java algorithm and are not altered by the reimplementation: the C++ port preserves the same objective, regularization, and background-sampling semantics, so switching implementations does not change the statistical behavior of the model:

- **Feature explosion.** The number of features (particularly hinge and threshold) grows with the number of environmental variables, which can cause identifiability issues in high-dimensional settings [8, 20].
- **Sensitivity to background sampling.** Maxent’s output is influenced by the choice and size of the background sample, which affects the estimated density and can bias predictions in undersampled regions [25].
- **ℓ_1 regularization limitations.** The sequential coordinate ascent with ℓ_1 penalties produces sparse models but does not guarantee selection of the “correct” features under high correlation among predictors [13].
- **No principled uncertainty quantification.** Unlike Bayesian approaches or bootstrap-based methods, the Maxent optimizer produces point estimates without confidence intervals.

A principled alternative would be to reformulate the Maxent objective using modern convex optimization tooling (e.g., proximal gradient methods, ADMM) or Bayesian inference. However, such reformulations would produce different results than the widely used Java implementation, breaking comparability with published analyses. The design of `maxentcpp` explicitly aims to preserve this comparability.

CRAN compliance and software quality

Achieving CRAN acceptance requires more than passing R CMD `check`: packages must meet strict software-quality standards that safeguard user environments and ensure reproducibility across platforms [4, 28]. `maxentcpp` (now on CRAN as version 1.0.0) was brought to compliance on four fronts. First, all 36 example blocks were migrated from `\dontrun{}` to `\donttest{}` [27] and rewritten to be self-contained with synthetic data, so every documented example is live, testable code that CRAN’s infrastructure can execute with `--run-donttest`. Second, functions that touch global graphics state now restore it through an internal `.maxent_safe_png()` helper built on `on.exit()`, guaranteeing cleanup even on error [28]. Third, all `terra`-dependent examples are guarded with `requireNamespace("terra", quietly = TRUE)` so they skip gracefully when the optional dependency is unavailable. Fourth, GitHub Actions runs R CMD `check --as-cran` on Ubuntu (R release and R-devel), macOS, and Windows, plus a CRAN-preflight job with `_R_CHECK_FORCE_SUGGESTS_=false` that simulates CRAN’s minimal-dependency environment. Memory safety is handled through Rcpp external pointers, which release C++ objects automatically when the corresponding R object is garbage-collected, and through `std::shared_ptr` ownership in the feature layer; the streaming provider maps tile data through `Eigen::Map` views to avoid copying raster blocks into the scoring engine. The cross-platform R CMD `check` matrix (Ubuntu release/devel, macOS, Windows) and the `donttest` examples, which double as runtime smoke tests, exercise the package under standard and edge-case inputs.

Research impact statement

maxentcpp provides a numerically faithful implementation of the core continuous-predictor Maxent fitting and projection workflow for the implemented feature classes and output transformations, including categorical predictors, replicate runs and cross-validation, jackknife diagnostics, and missing-data handling. SWD-to-raster projection remains a convenience workflow not yet exposed through the public R API.

The package is distributed under the MIT license (compatible with Eigen's MPL2 and RcppEigen's GPL-2 via the "or later" clause), with full source code, a pkgdown documentation website at <https://alroble.github.io/maxentcpp/>, and four vignettes covering a getting-started walkthrough, the mathematical foundations of Maxent features, a comparative motivation document, and a virtual species validation study. Continuous integration via GitHub Actions runs R CMD **check** on Ubuntu, macOS, and Windows across R release and R-devel. The test suite comprises over 20 test files (~4,800 lines) covering feature generation, optimization convergence, spatial projection, Java numerical equivalency, model diagnostics, and end-to-end workflows.

By providing a dependency-free, memory-efficient, and numerically faithful Maxent implementation, **maxentcpp** may lower the barrier for large-scale biodiversity assessments, climate-change projections, and conservation prioritization studies that rely on presence-only species distribution models, bringing the package to near-parity with Java Maxent 3.4.4 for the implemented feature classes while remaining installable in Java-free environments.

Software availability

maxentcpp is released under the MIT license (compatible with Eigen's MPL2 and RcppEigen's GPL-2 via the "or later" clause).

- **Package:** **maxentcpp** version 1.0.0 (CRAN); development version from GitHub via `remotes::install_github("alroble/maxentcpp")`
- **Source repository:** <https://github.com/alroble/maxentcpp>
- **Documentation:** pkgdown site at <https://alroble.github.io/maxentcpp/> (four vignettes: getting started, mathematical foundations, comparative motivation, virtual species validation)
- **Issue tracker:** <https://github.com/alroble/maxentcpp/issues>
- **Fidelity test suite:** companion package **maxentcppCompTest** (<https://github.com/alroble/maxentcppCompTest>)

AI usage disclosure

Generative AI tools were used during the development of **maxentcpp** and the preparation of this manuscript, as detailed below.

Tools used.

- **GitHub Copilot** (GPT-4-based, 2025 version): code scaffolding and boilerplate generation during the initial porting phases in **alroble/Maxent** and **alroble/maxentcpp-devel**.
- **Devin** (Cognition AI, 2025–2026): incremental feature implementation, test writing, documentation generation, CRAN compliance fixes, CI configuration, and manuscript drafting.

- **Claude** (Anthropic, 2025–2026): complex refactoring, cross-cutting documentation, and code review.

Nature and scope of assistance. AI tools contributed to: C++ code generation and refactoring from Java source, Rcpp binding scaffolding, R wrapper functions, `testthat` test scaffolding and expansion, `roxygen2` documentation, vignette drafting, pkgdown site configuration, CI workflow setup, CRAN compliance review, and manuscript drafting. The core algorithmic C++ classes (`Sequential`, `FeaturedSpace`, feature types) were translated from Java with AI assistance but under direct human supervision: each class was mapped line-by-line from the corresponding Java source, reviewed for correctness against the Java reference, and validated through the `maxentcppCompTest` trajectory comparison framework. The human author made all core design decisions, reviewed and edited all AI-generated code, and can independently explain and modify every algorithmically significant class (the optimizer methods are documented with line-by-line references to the corresponding Java source). The full development history is publicly available in the four repositories listed above.

Acknowledgements

The author thanks Steven J. Phillips, Robert P. Anderson, and Robert E. Schapire for creating the original Maxent software and making the source code publicly available under an MIT license. The `Rcpp`, `RcppEigen`, and `terra` package maintainers are gratefully acknowledged for providing the infrastructure that makes `maxentcpp` possible.

References

1. D. Bates and D. Eddelbuettel. Fast and elegant numerical linear algebra using the RcppEigen package. *Journal of Statistical Software*, 52(5):1–24, 2013.
2. J. Baumgartner. rmaxent: Tools for working with Maxent in R, 2017.
3. M. E. Cobos, A. T. Peterson, N. Barve, and L. Osorio-Olvera. kuenm: an R package for detailed development of ecological niche models using Maxent. *PeerJ*, 7:e6281, 2019.
4. CRAN Repository Maintainers. CRAN repository policy, 2024. Revision 6846.
5. D. Eddelbuettel. *Seamless R and C++ Integration with Rcpp*. Springer, New York, 2013.
6. D. Eddelbuettel and R. François. Rcpp: Seamless R and C++ integration. *Journal of Statistical Software*, 40(8):1–18, 2011.
7. J. Elith, M. Kearney, and S. Phillips. The art of modelling range-shifting species. *Methods in Ecology and Evolution*, 1:330–342, 2010.
8. J. Elith, S. J. Phillips, T. Hastie, M. Dudík, Y. E. Chee, and C. J. Yates. A statistical explanation of MaxEnt for ecologists. *Diversity and Distributions*, 17(1):43–57, 2011.
9. Á. L. R. Fernández. maxentcppCompTest: Cross-language fidelity tests for maxentcpp, 2025.
10. J. Friedman, T. Hastie, and R. Tibshirani. Regularization paths for generalized linear models via coordinate descent. *Journal of Statistical Software*, 33(1):1–22, 2010.

-
11. G. Guennebaud, B. Jacob, et al. Eigen v3, 2010.
 12. K. R. Hall, R. Anantharaman, V. A. Landau, M. Clark, B. G. Dickson, A. Jones, J. Platt, A. Edelman, and V. B. Shah. Circuitscape in Julia: Empowering dynamic approaches to connectivity assessment. *Land*, 10(3):301, 2021.
 13. T. Hastie, R. Tibshirani, and M. Wainwright. *Statistical Learning with Sparsity: The Lasso and Generalizations*. CRC Press, 2015.
 14. R. J. Hijmans. *terra: Spatial Data Analysis*, 2024. R package version 1.7-78.
 15. R. J. Hijmans, S. Phillips, J. Leathwick, and J. Elith. *dismo: Species Distribution Modeling*, 2023. R package version 1.3-14.
 16. E. T. Jaynes. Information theory and statistical mechanics. *Physical Review*, 106:620–630, 1957.
 17. J. M. Kass, R. Muscarella, P. J. Galante, C. L. Bohl, G. E. Pinilla-Buitrago, R. A. Boria, M. Soley-Guardia, and R. P. Anderson. ENMeval 2.0: Redesigned for customizable and reproducible modeling of species’ niches and distributions. *Methods in Ecology and Evolution*, 12(9):1602–1608, 2021.
 18. J. M. Kass, B. Vilela, M. E. Aiello-Lammens, R. Muscarella, C. Merow, and R. P. Anderson. wallace: A flexible platform for reproducible modeling of species niches and distributions built for community expansion. *Methods in Ecology and Evolution*, 9(4):1151–1156, 2018.
 19. B. Leroy, R. Delsol, B. Hugueny, C. N. Meynard, C. Baroni, et al. virtualspecies, an R package to generate virtual species distributions. *Ecography*, 39(6):599–607, 2016.
 20. C. Merow, M. J. Smith, and J. A. Silander, Jr. A practical guide to MaxEnt for modeling species’ distributions: what it does, and why inputs and settings matter. *Ecography*, 36(10):1058–1069, 2013.
 21. E. Pebesma. Simple features for R: Standardized support for spatial vector data. *The R Journal*, 10(1):439–446, 2018.
 22. S. J. Phillips. *maxnet: Fitting ‘Maxent’ Species Distribution Models with ‘glmnet’*, 2024. R package version 0.1.4.
 23. S. J. Phillips, R. P. Anderson, M. Dudík, R. E. Schapire, and M. E. Blair. Opening the black box: an open-source release of Maxent. *Ecography*, 40(7):887–893, 2017.
 24. S. J. Phillips, R. P. Anderson, and R. E. Schapire. Maximum entropy modeling of species geographic distributions. *Ecological Modelling*, 190(3–4):231–259, 2006.
 25. S. J. Phillips and M. Dudík. Modeling of species distributions with Maxent: new extensions and a comprehensive evaluation. *Ecography*, 31:161–175, 2008.
 26. S. J. Phillips, M. Dudík, and R. E. Schapire. A maximum entropy approach to species distribution modeling. In *Proceedings of the Twenty-First International Conference on Machine Learning, ICML ’04*, pages 655–662, New York, NY, USA, 2004. ACM.
 27. R Contributors. CRAN cookbook: General issues, 2024.
 28. R Core Team. *Writing R Extensions*, 2024. Version 4.4.0.
-

-
29. T. W. Schoener. The Anolis lizards of Bimini: resource partitioning in a complex fauna. *Ecology*, 49:704–726, 1968.
 30. W. Thuiller, D. Georges, and R. Engler. biomod2: Ensemble platform for species distribution modeling. *R package*, 2009.
 31. P. Tseng. Convergence of a block coordinate descent method for nondifferentiable minimization. *Journal of Optimization Theory and Applications*, 109:475–494, 2001.
 32. S. Vignali, A. G. Barras, R. Arlettaz, and V. Braunisch. SDMtune: An R package to tune and evaluate species distribution models. *Ecology and Evolution*, 10(20):11488–11506, 2020.
 33. J. Vollerling, R. Halvorsen, and I. G. Alsos. Measuring niche breadth and overlap with random modelling iterations: MIAMaxent. *Ecology and Evolution*, 9(23):13674–13687, 2019.
 34. D. L. Warren, R. E. Glor, and M. Turelli. Environmental niche equivalency versus conservatism: quantitative approaches to niche evolution. *Evolution*, 62:2868–2883, 2008.
 35. D. L. Warren, R. E. Glor, and M. Turelli. ENMTools: a toolbox for comparative studies of environmental niche models. *Ecography*, 33(3):607–611, 2010.

A Purpose and scope

This appendix reports the benchmark suite run to (i) answer reviewer requests for quantitative performance evidence (fit time, projection time, peak memory, scaling with raster size and sample size), and (ii) locate the computational bottlenecks of `maxentcpp` 1.0.0 so that the package can be optimized without breaking its numerical fidelity contract with Java Maxent 3.4.4.

Every number in this appendix was obtained by actually running the code on 11 August 2026. No figure is copied from an earlier draft or from the literature. The scripts that produced the numbers are listed in Section G together with their output files.

B Hardware and software environment

Component	Value
CPU	Intel Core Ultra 7 165H (16 cores / 22 threads, up to 5.0 GHz, heterogeneous P-core + E-core design)
RAM	62 GiB
OS	Ubuntu 24.04
R	4.6.0 (2026-04-24)
<code>maxentcpp</code>	1.0.0 (installed from CRAN)
<code>terra</code>	1.7.65
<code>maxnet</code>	0.1.4
Java	OpenJDK 21.0.11

Table 1. Benchmark environment.

Because the CPU is a heterogeneous design (performance cores and efficiency cores sharing a package), wall-clock timings show run-to-run variability of roughly $\pm 5\%$. All timings below are medians over at least three runs unless stated otherwise.

C Methodology

C.1 The in-place mutation artifact (critical)

`maxent_fit()` *mutates the `FeaturedSpace` object in place*. The optimizer updates the feature lambdas stored inside the featured space and leaves them there when it returns. Consequently, calling `maxent_fit()` twice on the *same* featured space does not measure two independent fits: the second call starts from the already-converged solution and terminates after only a handful of iterations (21 instead of 261 in our tests), producing a grossly optimistic timing.

This artifact is the explanation for the “~820 ms” figure that appeared in the v2 draft: it was measured by re-fitting a reused featured space. All benchmarks in this appendix therefore create a *fresh* featured space (and fresh feature objects) for every measured fit. Section D.1 shows the two numbers side by side so the difference is visible.

C.2 Warm-up

The first call to the Rcpp module in a fresh R process pays a one-time cost of roughly 5–19 s (library load / symbol resolution / page faults). Every benchmark below runs one or two warm-up fits first and excludes them from the reported timings.

C.3 Memory measurement

Peak resident set size (RSS) was measured with `/usr/bin/time -v` around a whole **Rscript** process that (a) performs a warm-up run, then (b) performs the measured fit and projection. Both `maxentcpp` and `maxnet` scripts load the same libraries and data so the comparison is apples-to-apples.

C.4 Data

Two datasets were used:

- **Bundled *Abeillia abeillei* dataset** (package `extdata`): 73 presence records, 2,371 non-NA background cells, two predictors (`bio1` = annual mean temperature, `bio12` = annual precipitation, Central America), linear + quadratic + hinge features, 44 features total, `max_iter` = 500, `convergence` = `1e-5`.
- **Synthetic grids** (for scaling and the 1.0.0 diagnostics): 2,371 / 23,000 / 233,000 cells with two continuous predictors (plus one five-level categorical for the diagnostics), Gaussian niches for presence sampling.

D Measured results

D.1 Fit time (*Abeillia* dataset)

The fresh-state fit converges in 261 iterations. The end-to-end `maxent_run()` adds evaluation (AUC), percent contribution, permutation importance, and CSV output; each of those stages costs 2–3 ms, so the optimizer itself accounts for more than 99% of the wall time (Section D.2).

Measurement	maxentcpp	maxnet
Fresh-state fit, median of 4 runs	18.40 / 18.80 / 18.88 / 18.44 s	—
Fresh-state fit, median	~18.5 s	—
Warm fit, median of 3 runs	~0.25 s	~0.25 s
Iterations to convergence (fresh)	261	—
End-to-end <code>maxent_run()</code> (fit+eval+diag), median of 3	32.3 s	—

Table 2. Fit time on the bundled *Abeillia abeillei* dataset. The “warm fit” row for maxentcpp reuses a converged featured space and is shown only to demonstrate the in-place artifact; it is not a valid per-fit cost.

Stage	Wall time
Grid info + dimension	<1 ms
Read occurrences	1 ms
Background indices (n = 10,000)	1 ms
Extract env values + complete cases	3 ms
Generate features (44)	<1 ms
Featured space + fit (261 iters)	18,053 ms
Evaluate (pres + bg predictions, AUC)	3 ms
Percent contribution	2 ms
Permutation importance	3 ms

Table 3. Stage-by-stage breakdown of `maxent_run()`. The fit dominates: 18,053 ms of 18,060 ms total.

D.2 Time breakdown

A step-by-step trace of `maxent_run()` on the Abeillia dataset (2,371 cells, 73 presences, 44 features):

This is the single most important result for optimization: **any speedup of the sequential optimizer translates almost 1:1 into end-to-end speedup**; speeding up every other stage combined cannot save more than ~1%.

D.3 Peak memory (RSS)

Workflow	Peak RSS
maxentcpp <code>maxent_run()</code> + full-raster projection	158 MB (162,076 KB)
maxnet fit + predict (same data)	358 MB (366,176 KB)

Table 4. Peak resident set size measured with `/usr/bin/time -v` on the same 2,371-cell dataset, warm-up excluded. maxentcpp uses ~2.3× less peak memory.

The memory advantage is real and already quantified. It comes from streaming evaluation (tiles are scored and discarded) and from not materializing the full prediction matrix. Optimization work should *preserve* this property.

D.4 Projection time

D.5 Scaling with sample size

The 23K / 2,371 time ratio is ~7× for ~9.7× more cells, i.e. roughly linear in the number of cells (the sub-linear ratio is explained by the two runs converging in different numbers of iterations). Per-iteration cost scales linearly with the number of points, as expected from the hot loops described in Section E.

Grid	maxentcpp	maxnet
23,000 cells	21 ms	9 ms
Full raster, 6,930 cells (incl. NODATA)	3 ms	—

Table 5. Projection time. Both implementations are trivial at these scales; the streaming advantage only matters for rasters larger than RAM (future work on this hardware).

Cells	maxentcpp fresh fit	Notes
2,371	~18.1–19.6 s	converges in 261 iters
23,000	~129 s	exhausts 500-iteration budget
233,000	not run	exceeds per-session budget

Table 6. Scaling of fresh-state fit time with raster size. The 23K run uses signal-strong synthetic data that exhausts `max_iter`; real-world data converges earlier.

D.6 1.0.0 diagnostics suite

Benchmarked on a synthetic grid of 2,371 cells, 100 presence records, two continuous predictors plus one five-level categorical variable, lqh features, `max_iter` = 500, fresh state per fit:

Diagnostic	Wall time
Single fit (continuous only)	~22 s
Fit with categorical predictor	~18 s
Jackknife (3 variables; 6 fits)	~63 s
Cross-validation ($k = 5$; 5 fits)	~135 s
Replicate runs (bootstrap, $n = 5$; 5 fits)	~102 s

Table 7. 1.0.0 diagnostics. Each diagnostic is a small multiple of a single fit, as expected. The previously published “~1 ms / ~94 ms” figures were in-place-mutation artifacts.

D.7 Expanded virtual-species simulation

48 models: 2 predictor-correlation structures ($\rho = 0, 0.8$) $\times$ 4 Gaussian virtual species (narrow/wide $\times$ centered/offset) $\times$ 2 sample sizes (100, 400) $\times$ 3 replicates. Every model compared against its true suitability surface and against the other package.

D.8 Java Maxent prediction agreement

Direct prediction comparison between Java Maxent 3.4.4 (`MaxentJavaRunner.trainLinear2Var` harness) and `maxentcpp` on identical data (2,100 cells, 100 presences, linear features, `max_iter` = 500, `convergence` = `1e-5`, `beta` = `1.0`, `min_deviation` = `0.001`):

E Hot paths in the optimizer

The sequential optimizer lives in `src/cpp/include/maxent/sequential.hpp` (class `Sequential`, mirroring Java `density.Sequential`). The measured numbers (261 iterations, ~18.1 s, 2,371 points, 44 features) imply a per-iteration cost of

$$18.1 \text{ s} / 261 \approx 69 \text{ ms/iteration.}$$

Three loops dominate that budget:

Quantity	Value
Agreement between packages (r), median	0.98 (min 0.95 across all 48 cells)
r maxentcpp vs true, median	0.88
r maxnet vs true, median	0.94
Mean $ \Delta\text{cloglog} $, median	0.044
Correlation effect on agreement ($\rho = 0$ vs $\rho = 0.8$)	0.985 vs 0.973
maxentcpp fit, median	34.8 s
maxnet fit, median	0.94 s
maxentcpp iterations, median	141

Table 8. Simulation summary. The statistical results justify the equivalence claims in the paper; the timing rows quantify the performance gap on synthetic Gaussian niches.

Metric	Value
Pearson r	1.000000
Spearman ρ	1.000000
RMSE	9.18×10^{-18}
Mean $ \Delta\text{cloglog} $	4.15×10^{-18}
Max $ \Delta\text{cloglog} $	8.76×10^{-17}
AUC (both)	0.903525

Table 9. Java vs C++ prediction agreement — machine precision.

1. `get_best_feature()` — **feature selection.** Every non-parallel iteration scans all $J = 44$ features and calls `delta_loss_bound(j)` for each. Each bound is $O(n)$ in the number of points $n = 2,444$, so this step is $O(n \times J)$ per iteration. With 261 iterations this is the dominant cost.
2. `search_alpha()` / `new_loss_change()` — **step-size search.** After selecting a feature, the optimizer computes a Newton step and then searches for a better step size by repeatedly calling `new_loss_change(j, alpha)`, each call looping over all n points with a `std::exp()` per point. The quadrupling loop can evaluate this several times per update.
3. `do_parallel_update()` — **every 10 iterations.** Computes the Hessian-vector product and gradient over all n points for the subset of features scheduled for update ($O(n \times n_{\text{update}})$).

All three loops are single-threaded by design: the optimizer must preserve Java Maxent’s exact iteration order for the fidelity contract (`maxentcppCompTest` compares per-iteration loss, entropy, and λ trajectories). Any optimization must therefore preserve bit-comparable results under the deterministic settings, or be gated behind an opt-in flag with the fidelity suite re-run.

F Optimization opportunities (for the developer)

The following directions are ordered by expected impact. They are *suggestions grounded in the measured hot paths above*, not already-verified changes; each must be validated against `maxentcppCompTest` and the benchmark scripts.

F.1 High impact

1. **Vectorize the $O(n)$ loops with Eigen.** `new_loss_change()` and `delta_loss_bound()` currently loop over points with scalar `std::exp()` calls.

`std::exp()` on a vectorized Eigen array is $\sim 4\text{--}8\times$ faster on modern x86 (the hardware has AVX2). The package already depends on Eigen (used for the feature matrix and streaming provider), so this is a pure implementation change with no algorithm change.

2. **Avoid recomputing per-feature statistics.** `get_best_feature()` recomputes `delta_loss_bound` for *all* features every iteration, even though only the selected feature's lambda changed. Caching per-feature bounds and updating only the affected entries would reduce the selection scan from $O(n \times J)$ to $O(n \times k)$ where $k \ll J$ is the number of recently changed features. The Java code has a `featuresToUpdate` notion; the C++ port already computes `state_[j].last_change` — the cache can key on that.
3. **Reduce search_alpha evaluations.** The quadrupling loop calls `new_loss_change` (an $O(n)$ pass with one `exp` per point) until the loss stops improving. A golden-section or analytic bound on the quadratic would cut the number of passes, or the exp computations of successive trials could be reused (they differ only by the step multiplier).

F.2 Medium impact

1. **Parallelize across independent fits.** Replicate runs, cross-validation folds, and jackknife leave-one-out fits are independent by construction. The paper already states parallelism is applied “across independent fits”; making it real with OpenMP/threads (or documented `future` support) would turn the 63–135 s diagnostics into 63/cores–135/cores seconds with zero fidelity risk, because each fit is bit-identical to the serial one.
2. **Compile flags.** Verify the package builds with `-O3` and, where CRAN permits, `-march=native` on Linux; profile-guided optimization for the hot loops. (CRAN's policy restricts `-march` on CRAN builds; the effect can be delivered to users via a documented `Makevars` override without changing the default.)
3. **Reduce `std::exp` precision where safe.** `new_loss_change` accumulates `exp(alpha * F(i,j))`. If the fidelity suite confirms the summation order is the only sensitive part, an opt-in `-ffast-math`-style fast exp (or `expf` with double accumulation) could be gated behind an option, but this is risky and must be off by default.

F.3 Low impact / do not touch

1. **Streaming projection.** Already 21 ms at 23K cells and memory-efficient; there is nothing to gain at these scales.
2. **Feature generation.** Measured at <1 ms; not a bottleneck.
3. **The fidelity contract.** The per-iteration trajectory tests are the package's core value proposition; never change the default algorithm path in a way that alters iteration order or numerical results.

G Reproducibility

All scripts and raw outputs used for this appendix are kept under `/tmp/javabench/` on the benchmark machine (Gamma):

-
- `fresh_test.R` — fresh-state fit timings (4 runs, 261 iterations each)
 - `desglose2.R` / `run_trace.R` — stage-by-stage breakdown of `maxent_run()`
 - `mem_maxentcpp.R` / `mem_maxnet.R` — RSS under `/usr/bin/time -v` (outputs `mem_cpp.txt`, `mem_mn.txt`)
 - `diag_bench.R` — 1.0.0 diagnostics timings (output `diag_bench_out.txt`)
 - `sim_major3.R` — expanded simulation (48 cells; outputs `sim_major3.csv`, `sim_major3.rds`)
 - `sim_figure.R` — `figures/simulation_summary.png`
 - `compare_java_cpp.R` — Java vs C++ prediction comparison (output `comparison.rds`, `figures/java_vs_cpp_predictions.png`)
 - `appendix_timing.R` — clean end-to-end `maxent_run()` timing (output `appendix_timing_out.txt`)

To re-run a single measurement, copy the script to a machine with `maxentcpp` 1.0.0 installed and execute `Rscript <script>.R`. The simulation requires 4 cores for `mclapply`; all other scripts are single-process.

H Summary for the optimization task

- The optimizer is >99% of runtime. Optimize `sequential.hpp` only.
- Target: reduce the ~69 ms/iteration on 2,371 points / 44 features. The three hot loops are feature selection ($O(nJ)$), step-size search ($O(n)$ with multiple `exp()` passes), and the periodic parallel update.
- The fastest safe wins: Eigen-vectorized `exp` loops and caching of `delta_loss_bound` between iterations.
- Parallelism across independent fits (replicates / CV / jackknife) is free of fidelity risk.
- Keep peak RSS at or below ~158 MB on this dataset — it is currently a 2.3× advantage over `maxnet`.
- After any change, re-run `maxentcppCompTest` and the scripts above; the fidelity suite is the gate.